

Продвинутое использование git

Лабораторная работа № 4

Чжао Синья (zhaoxinya)

2026-10-07

Продвинутое использование git

Лабораторная работа № 4

Студент: Чжао Синья

Учётная запись: zhaoxinya

Номер студенческого билета: 113225832

Дисциплина: Архитектура компьютеров и операционные системы.

Операционные системы

Цель работы

Получение навыков правильной работы с репозиториями git.

- › Освоение методологии Gitflow
- › Освоение семантического версионирования
- › Освоение общепринятых коммитов (Conventional Commits)

Задание работы

1. Установить необходимое ПО (git-flow, Node.js, pnpm, commitizen, conventional-changelog)
2. Создать репозиторий git-extended на GitHub
3. Настроить и применить Conventional Commits
4. Освоить рабочий процесс Gitflow
5. Применить семантическое версионирование
5. Создать релизы 1.0.0 и 1.2.3 с генерацией CHANGELOG

Рабочий процесс Gitflow

Gitflow Workflow — модель ветвления, предложенная Винсентом Дриссеном.

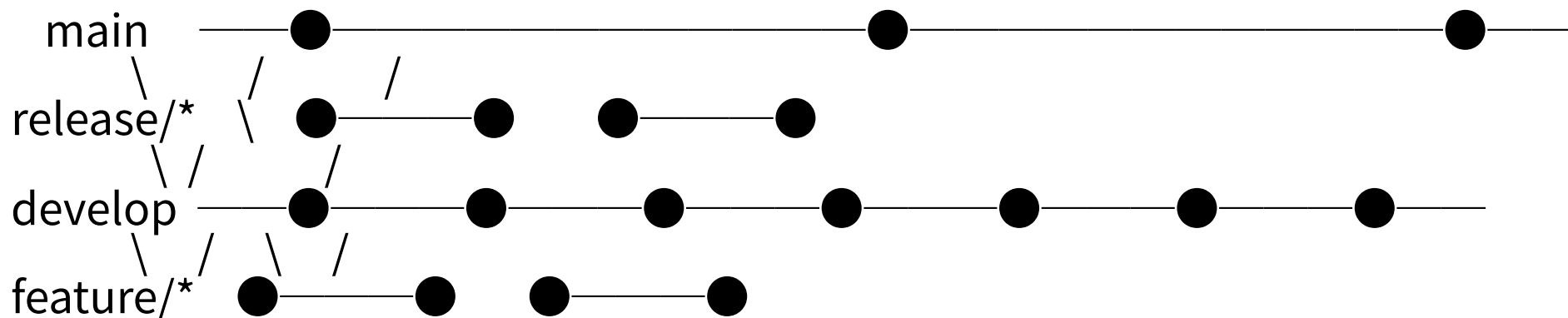
Основные ветки

- › main — официальная история релизов
- › develop — интеграция всех новых функций

Вспомогательные ветки

- › feature/* — разработка отдельных функций
- › release/* — подготовка релиза
- › hotfix/* — срочные исправления

Gitflow: схема ветвления



- › feature → develop
- › release → main + develop
- › hotfix → main + develop

Семантическое версионирование

Формат версии:

МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ

- › МАЖОРНАЯ — несовместимые изменения API
- › МИНОРНАЯ — новая функциональность без нарушения совместимости
- › ПАТЧ — обратно совместимые исправления

Пример: 1.2.3

Conventional Commits

Структура коммита

<тип>(<область>): <описание изменения>

[необязательное тело]

[необязательный нижний колонтитул]

Основные типы

- › feat — новая функция (MINOR)
- › fix — исправление ошибки (PATCH)
- › docs — документация
- › chore — служебные изменения
- › refactor, style, test, perf, build, ci

Связь Conventional Commits и SemVer

Тип коммита	Версия
-------------	--------

fix	PATCH
-----	-------

feat	MINOR
------	-------

BREAKING CHANGE	MAJOR
-----------------	-------

Позволяет автоматически генерировать:

- › номера версий
- › CHANGELOG

Соглашения об именовании

- › Пользователь ВМ: zhaoxinya
- › Имя хоста: zhaoxinya
- › Имя виртуальной машины: ZhaoXinya2
- › Репозиторий эксперимента: git-extended
- › Основной курсовой репозиторий: study_202602-202606_os-intro

Проверка:

whoami

hostname

id

Установка программного обеспечения

git-flow через COPR

```
sudo dnf -y copr enable elegos/gitflow
```

```
sudo dnf -y install gitflow
```

Node.js и pnpm

```
sudo dnf -y install nodejs npm pnpm
```

Настройка pnpm

```
pnpm setup
```

```
source ~/.bashrc
```

commitizen и conventional-changelog

```
pnpm add -g commitizen
```

```
pnpm add -g conventional-changelog-cli
```

Проверка версий инструментов

git version 2.49.0
git flow version 1.12.3 (AVH Edition)
gh version 2.87.3
node v22.22.0
npm 10.9.4
pnpm 12.9.1

Создание репозитория git-extended

```
mkdir -p ~/work/git-extended  
cd ~/work/git-extended  
git init
```

```
gh repo create git-extended --public \  
--description "Git repo for educational purposes" \  
--source=. --remote=origin
```

Результат:

- ✓ Created repository Muzxnya/git-extended
- ✓ Added remote git@github.com:Muzxnya/git-extended.git

package.json

```
{  
  "name": "git-extended",  
  "version": "1.0.0",  
  "description": "Git repo for educational purposes",  
  "main": "index.js",  
  "author": "zhaoxinya <1132258332@rudn.ru>",  
  "license": "CC-BY-4.0",  
  "config": {  
    "commitizen": {  
      "path": "cz-conventional-changelog"  
    }  
  }  
}
```

Первый conventional commit

`git add .`

`git cz`

Выбор:

- › type: feat
- › scope: пропущено
- › short description: initial commit

Результат:

[main (root-commit) 928c072] feat: initial commit

Инициализация git-flow

`git flow init`

Параметры:

- › main: main
- › develop: develop
- › feature: feature/
- › release: release/
- › hotfix: hotfix/
- › tag prefix: v

Публикация:

`git push --all`

Релиз 1.0.0

```
git flow release start 1.0.0  
conventional-changelog -p angular -r 0 > CHANGELOG.md  
git add CHANGELOG.md  
git commit -m "chore(site): add changelog"  
git flow release finish 1.0.0  
git push --all  
git push --tags  
gh release create v1.0.0 -F CHANGELOG.md -t "Release 1.0.0"
```

CHANGELOG.md (1.0.0)

1.0.0 (2026-10-07)

Features

- * initial commit ([928c072](...))

Разработка новой функциональности

```
git flow feature start feature_branch  
echo "New feature content" > feature.txt  
git add feature.txt  
git cz  
git flow feature finish feature_branch  
Коммит:  
[develop 7da1cc4] feat: add new feature.txt
```

Релиз 1.2.3

`git flow release start 1.2.3`

обновить версию в package.json → 1.2.3

`conventional-changelog -p angular -r 0 > CHANGELOG.md`

`git add CHANGELOG.md package.json`

`git commit -m "chore(site): update changelog and version"`

`git flow release finish 1.2.3`

Конфликт слияния

При git flow release finish 1.2.3 возник конфликт:

CONFLICT (add/add): Merge conflict in CHANGELOG.md

Automatic merge failed; fix conflicts and then commit the result.

Разрешение:

- › вручную удалены маркеры <<<<<<, =====, >>>>>>
- › оставлены обе строки Features

```
git add CHANGELOG.md
```

```
git commit -m "Merge branch 'release/1.2.3'"
```

CHANGELOG.md (1.2.3)

1.2.3 (2026-10-07)

Features

- * add new feature.txt ([7da1cc4](...))
- * initial commit ([928c072](...))

Публикация релиза 1.2.3

```
git tag -a v1.2.3 -m "Release 1.2.3"
```

```
git push --all
```

```
git push --tags
```

```
gh release create v1.2.3 -F CHANGELOG.md -t "Release 1.2.3"
```

Результат:

<https://github.com/Muzxnya/git-extended/releases/tag/v1.2.3>

Список релизов

TITLE	TYPE	TAG NAME	PUBLISHED
Release 1.2.3	Latest	v1.2.3	about 1 minute ago
Release 1.0.0		v1.0.0	about 30 minutes ago

Выводы

- › Освоена методология Gitflow
 - › Изучена спецификация Conventional Commits
 - › Освоено семантическое версионирование
 - › Применена автоматическая генерация CHANGELOG
 - › Освоена утилита gh для работы с GitHub
 - › Получены навыки разрешения конфликтов слияния
- Все релизы опубликованы в репозитории git-extended.

Ответы на контрольные вопросы (1/3)

1. Что такое Gitflow?

Модель ветвления Git, предложенная Винсентом Дриссенем. Использует две долгоживущие ветки (main, develop) и временные (feature, release, hotfix).

2. Какие основные ветки используются в Gitflow?

- › main — история релизов
- › develop — интеграция функций
- › feature/* — разработка
- › release/* — подготовка релиза
- › hotfix/* — срочные исправления

Ответы на контрольные вопросы (2/3)

3. Что такое семантическое версионирование?

Система нумерации МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ:

- › МАЖОРНАЯ — несовместимые изменения API
- › МИНОРНАЯ — новая функциональность
- › ПАТЧ — обратно совместимые исправления

4. Что такое Conventional Commits?

Спецификация структуры сообщений коммитов:

<тип>(<область>): <описание>

Ответы на контрольные вопросы (3/3)

5. Связь Conventional Commits и SemVer?

fix → PATCH, feat → MINOR, BREAKING CHANGE → MAJOR.

6. Что делает git flow init?

Инициализирует структуру Gitflow, создаёт develop, настраивает префиксы.

7. Что делает git flow release start 1.0.0?

Создаёт ветку release/1.0.0 на основе develop.

8. Что делает git flow release finish 1.0.0?

Сливает релиз в main и develop, создаёт тег, удаляет релизную ветку.

9. Что такое CHANGELOG?

Файл со списком изменений по версиям.

10. Как создать релиз на GitHub?

```
gh release create v1.0.0 -F CHANGELOG.md -t "Release 1.0.0"
```

Спасибо за внимание!

Репозиторий:

<https://github.com/Muzxnya/git-extended>

Speaker notes